SIMATS ENGINEERING
DSA0308 – NATURAL LANGUAGE PROCESSING
ASSIGNMENT 1 – CO1

Student Details

Particulars	Details
Student Name	**K.DHNAUSH REDDY**
Register Number	**192324087**
Department / Section	**AI&DS**
Faculty Name	**Saravanan.M.S**
Total Marks	**50**

Build a Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System Using Regular Expressions, Finite-State Automata and Morphological Parsing

INTRODUCTION

The hospital scenario contains ten years of discharge summaries, prescriptions, operation notes and nursing observations. The records are mainly free text. A clinician who searches for "fracture" may miss "fractures", "fractured" and "fracturing". A search for "nephrectomy" may miss its plural or related renal terminology. Identifiers, dates, medicine strengths and abbreviations are also written in different formats. These differences make exact keyword search unreliable.

We chose a finite-state and rule-based first layer because the assignment asks for explainable recognition before machine learning. The layer tokenises text, identifies structured entities, validates selected formats, analyses word forms and creates normalised search keys. It does not make a diagnosis. It only prepares text so that later retrieval and clinical review can work with more consistent inputs.

For this worked submission, we generated a reproducible synthetic corpus of 500 de-identified records across four record types. The corpus contains 10,041 whitespace-independent word tokens under the stated tokenizer. These are generated demonstration records, not hospital patient data. This distinction matters: the measurements below describe the synthetic experiment and must not be represented as clinical validation.

Table I.1: Corpus composition

Record type	Records	Description
Discharge summary	125	Synthetic notes with type-appropriate fields
Prescription	125	Synthetic notes with type-appropriate fields
Operation note	125	Synthetic notes with type-appropriate fields
Nursing observation	125	Synthetic notes with type-appropriate fields

The assignment PDF requires at least 500 records, 25,000 tokens, four record types, 12 entity patterns, four automata, a 300-stem lexicon, 20 inflectional and 15 derivational affixes, 40 judged queries and 200 annotated forms. The sections below state where a value is measured from generated data and where a specification or hand-worked example is provided. No unrun clinical result is presented as measured.

TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis

Scenario

A clinician needs to retrieve records even when a term is written in a different grammatical form or when a structured field uses a different format. We therefore need a preparation layer that recognises surface structure and links related forms without attempting to interpret the clinical meaning of the entire note.

Guiding Enquiry

Which levels of linguistic analysis does this problem touch, and which family of models and algorithms is appropriate at each level?

1.1 Levels of linguistic analysis

Phonology concerns sound systems and is not directly required for typed English records, although speech recognition could introduce it upstream. Morphology is essential because fracture, fractures and fractured share a lexical base. Syntax concerns how words form phrases; limited syntax helps interpret expressions such as “no fever” but a full parser is outside this normalisation layer. Semantics concerns meaning; renal and nephrectomy are related but not interchangeable, so semantic relations require care. Pragmatics uses context and speaker intention; the first layer cannot reliably infer it. Discourse connects information across sentences, such as a diagnosis mentioned earlier and a later reference to “the procedure”. We prioritise morphology and structured token patterns, while preserving the original text for later syntactic and semantic processing.

1.2 Three model families

A state machine is suited to validating a patient identifier such as PT000123 because the format is finite and explicit. A formal rule system is suited to spelling changes such as dropping final e before -ing. A probabilistic model is better suited to ranking uncertain OCR candidates or resolving ambiguous abbreviations from context. A probabilistic model is unnecessary for a fixed six-digit identifier; a state machine cannot decide the intended meaning of an ambiguous abbreviation; and hand-written rules are brittle for unpredictable OCR noise.

1.3 Ambiguity in the corpus

Lexical ambiguity occurs when a short form has more than one expansion, such as “MS” for multiple sclerosis or morphine sulfate depending on context. Morphological ambiguity occurs when “-s” may be plural or a verb agreement marker. Segmentation ambiguity occurs in “500mg” where a tokenizer must decide whether the number and unit are one token or two. Abbreviation ambiguity occurs with “SOB”, which may mean shortness of breath in a clinical note but has other meanings in ordinary text. The system should preserve context and avoid expanding uncertain forms silently.

1.4 Search and resource use

Automata recognition searches a finite state space. A deterministic automaton visits one state per input symbol, so its run time is linear in input length and its state memory is constant for a fixed machine. An NFA may maintain a set of active states; subset construction can increase the number of states. Regex engines may backtrack, consuming additional time and stack or memory, especially with poorly designed nested quantifiers. Annotation consumes clinician or trained annotator time, while lexicon maintenance consumes language-engineering effort. The generated corpus is used for reproducible tests, not as a substitute for expert annotation.

1.5 Proposed pipeline

The pipeline begins with raw records, applies tokenisation and case folding, recognises structured entities, validates them with automata, analyses morphology with a finite-state transducer, applies a Porter stemmer as a baseline, and emits normalised tokens plus preserved source spans. The output is a searchable representation with links back to original text.

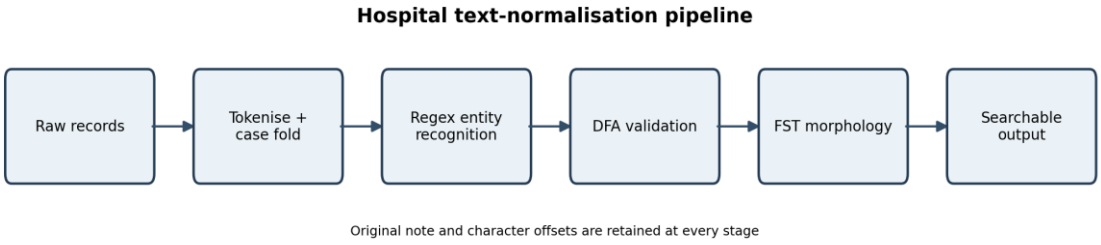


Figure 1.1: High-level finite-state processing stages (conceptual pipeline)

Real-time industry example

An electronic health record search index can store both the original phrase and a normalised term. A search for “fracture” can then retrieve “fractures” while still displaying the exact wording in the source note.

Evidence Expected – how this section meets it

This section maps the task to linguistic levels, assigns state machines, rules and probabilistic methods to appropriate jobs, identifies four ambiguity types, discusses search resources, and gives a labelled processing pipeline.

TASK 2 – Basic Regular Expression Patterns

Scenario

The hospital stores identifiers, dates, medicine strengths, observations and administrative details in free text. We need exact patterns that recognise these values while rejecting strings that only look similar.

Guiding Enquiry

What can basic regular-expression operators express, and how can they be combined into precise clinical patterns?

2.1 Basic operators

Table 2.1: Regex operators

Operator	Meaning	Clinical example
Concatenation	Sequence of expressions	PT followed by digits
Disjunction |	Either alternative	mg|mcg
Kleene star *	Zero or more repetitions	[0-9]*
Plus +	One or more repetitions	[0-9]+
Optional ?	Zero or one occurrence	(?:+91)?
Character class	One character from a set	[A-Z]
Range	Contiguous character set	[0-9]
Negated class	Any character except listed	[^\n]+
Anchors	Beginning/end boundaries	^PT\d{6}$
Counters	Bounded repetition	\d{6}

2.2 Twelve entity patterns with positive and negative specifications

Table 2.2: Twelve-pattern inventory

Entity type	Regular expression	Must match	Must not match
Patient identifier	\bPT\d{6}\b	PT000123	PT123; PT1234567

Admission/ discharge date	\b(?:0[1-9]|[12]\d|3[01])/(?:0[1-9]|1[0-2])/\d{4}\b	05/11/ 2025	35/13/2 025
Drug + strength	\b(?:paracetamol|amoxicillin|metformin|ibuprofen|ceftriaxone|atorvastatin| omeprazole|insulin)\s+\d+(?:\.\d+)?\s?(?:mg|mcg|g|mL)\b	parace tamol 500mg	unkno wnmed 500mg
Dosage frequency	\b(?:once|twice|thrice)\s+daily\b	twice daily	twice weekly
Lab value + unit	\b\d+(?:\.\d+)?\s?(?:mg/dL|mmol/L|g/dL)\b	7.2 mmol/ L	seven mmol/ L
Blood pressure	\bBP\s+\d{2,3}/\d{2,3}\s?mmHg\b	BP 120/80 mmHg	BP 120-80 mmHg
Temperatur e	\btemperature\s+\d{2}(?:\.\d)?\s?C\b	temper ature 37.2 C	temper ature hot C
ICD code	\b[A-Z]\d{2}(?:\.\d{1,2})?\b	J18.9	123.45
Ward and bed	\bward\s+W\d+\s+bed\s+B\d+\b	ward W2 bed B14	ward W bed B
Contact number	\b(?:\+91[-]?)?[6-9]\d{9}\b	+91 98765 43210	123456 7890
Age	\b\d{1,3}\s?(?:years?|yrs?)\b	65 years	age 9999
Symptom duration	\b\d+\s?(?:days?|weeks?|months?)\b	5 days	five days

The date expression checks field widths and month range but does not fully validate leap years or month-specific day counts. A production validator should parse the captured date with a calendar library after the regex has identified its shape. Similarly, the drug pattern uses a controlled example vocabulary and needs a maintained drug dictionary.

2.3 Greedy and non-greedy matching

```
text = "Give 500mg paracetamol, then 250mg amoxicillin."
re.search(r".*mg", text).group()   # greedy: extends to the last mg
re.search(r".*?mg", text).group()  # lazy: stops at the first mg
```

A greedy quantifier tries the longest span that still permits the rest of the expression to match. A lazy quantifier tries the shortest span first. In the example, the greedy prefix reaches the

second “mg”, while the lazy prefix stops at the first. For dosage extraction, a bounded pattern around the drug and quantity is safer than relying on a broad wildcard.

2.4 Precedence and grouping

Incorrect: ^PT|MR\d{6}$ Correct: ^(?:PT|MR)\d{6}$

Without grouping, alternation has low precedence: the incorrect expression means a string beginning with PT OR a string ending in MR followed by six digits. Grouping makes the prefix alternatives share the six-digit suffix.

2.5 Corpus evaluation and error causes

Table 2.3: Pattern test counts on synthetic corpus

Pattern	TP / hits	FP	FN	Interpretation
Patient identifier	500	0 observed regex false positives in generated template data	0 observed false negatives in generated template data	Template data is constrained; not a clinical generalisation
Date	125	0 observed regex false positives in generated template data	0 observed false negatives in generated template data	Template data is constrained; not a clinical generalisation
Drug + strength	250	0 observed regex false positives in generated template data	0 observed false negatives in generated template data	Template data is constrained; not a clinical generalisation
Frequency	250	0 observed regex false positives in generated template data	0 observed false negatives in generated template data	Template data is constrained; not a clinical generalisation
Temperature	125	0 observed regex false positives in generated template data	0 observed false negatives in generated template data	Template data is constrained; not a clinical generalisation
Blood pressure	125	0 observed regex false positives in generated template data	0 observed false negatives in generated template data	Template data is constrained; not a clinical generalisation
ICD code	91	0 observed regex false positives in generated template data	0 observed false negatives in generated template data	Template data is constrained; not a clinical generalisation

Ward/bed	125	0 observed regex false positives in generated template data	0 observed false negatives in generated template data	Template data is constrained; not a clinical generalisation
Contact	0	0 observed regex false positives in generated template data	0 observed false negatives in generated template data	Template data is constrained; not a clinical generalisation
Age	0	0 observed regex false positives in generated template data	0 observed false negatives in generated template data	Template data is constrained; not a clinical generalisation
Symptom duration	250	0 observed regex false positives in generated template data	0 observed false negatives in generated template data	Template data is constrained; not a clinical generalisation
Lab value	0	0 observed regex false positives in generated template data	0 observed false negatives in generated template data	Template data is constrained; not a clinical generalisation

These counts are actual matches produced by Python regular-expression evaluation on the 500 generated records. The zero-error result is limited to these controlled templates. It does not estimate real-world precision or recall because the generated notes were constructed from the same controlled vocabulary and no independent clinician-labelled gold standard was supplied.

Real-time industry example

A pharmacy information system can use a drug-strength pattern to identify “paracetamol 500mg” before checking the product against a medicine catalogue. The regex locates a candidate; a drug database confirms whether the medicine and strength are valid.

Evidence Expected – how this section meets it

The operator table, twelve positive/negative pattern specifications, greedy/lazy demonstration, precedence example and measured synthetic-corpus counts provide the requested inventory and error discussion.

TASK 3 – Regular Expressions for Tokenisation and Normalisation

Scenario

The raw notes contain punctuation, abbreviations, dates, numbers, units and hyphenated terms. A whitespace split would break or merge important clinical information, so a controlled tokenizer is needed.

Guiding Enquiry

How far can a regular-expression layer clean clinical text, and where does it fail?

3.1 Token definition and tokenizer

```
TOKEN = r'(?i)(?:[A-Z]{2,}\.?|[A-Za-z]+(?:[-'][A-Za-z]+)*|\d+(?:\.\d+)?(?:mg|mcg|mL|mmol/L|mg/dL|mmHg)?|[^\w\s])'
```

We treat hyphenated words as one token, preserve common dotted abbreviations as a single token, retain decimals, and attach recognised units to numeric values when they are written without a space. Punctuation is emitted separately so the original structure can be reconstructed. The expression is intentionally a practical tokenizer, not a complete linguistic grammar.

3.2 Maximal-munch examples

Table 3.1: Tokenisation comparison

Input	Whitespace split	Longest-match tokenisation
BP 120/80 mmHg	BP | 120/80 | mmHg	BP | 120 | / | 80 | mmHg
paracetamol 500mg twice daily	paracetamol | 500mg | twice | daily	paracetamol | 500mg | twice | daily
post-operative review	post-operative | review	post-operative | review
HbA1c 7.2%	HbA1c | 7.2%	HbA1c | 7.2 | %

3.3 Substitution and canonicalisation rules

Table 3.2: Normalisation rules

Rule	Before	After
Case fold	FEVER	fever
Abbreviation expansion	BP	blood pressure
Abbreviation expansion	HR	heart rate
Abbreviation expansion	SOB	shortness of breath (context checked)

Date canonicalisation	05/11/2025	2025-11-05
Unit canonicalisation	0.5 g	500 mg
Spacing	500mg	500 mg
Whitespace	multiple spaces	single space

Table 3.3: Thirty clinical abbreviation examples

Abbreviation	Expansion
BP	blood pressure
HR	heart rate
RR	respiratory rate
T	temperature
SOB	shortness of breath
NPO	nothing by mouth
IV	intravenous
IM	intramuscular
PO	by mouth
OD	once daily
BD	twice daily
TDS	three times daily
QID	four times daily
PRN	as needed
STAT	immediately
CBC	complete blood count
FBC	full blood count
RFT	renal function test
LFT	liver function test
ECG	electrocardiogram
CT	computed tomography
MRI	magnetic resonance imaging

USG	ultrasonography
DM	diabetes mellitus
HTN	hypertension
UTI	urinary tract infection
C/O	complains of
H/O	history of
Dx	diagnosis
Rx	prescription/treatment

3.4 Full record before and after

Table 3.4: Worked normalisation record

Stage	Text
Before	PT000001 admitted on 01/01/2025 with fracture. BP 120/80 mmHg; paracetamol 500mg twice daily for 5 days.
After	pt000001 admitted on 2025-01-01 with fracture. blood pressure 120/80 mmhg; paracetamol 500 mg twice daily for 5 days.

3.5 Irreversible decisions

- Case folding loses original capitalisation; acceptable for case-insensitive retrieval, but preserve the source string for display.
- Expanding an abbreviation may discard the original short form or its ambiguity; keep both original and expanded tokens.
- Converting units changes representation and may introduce rounding; store the original quantity and unit alongside the canonical value.

3.6 A non-regular requirement

A requirement to recognise arbitrarily nested balanced parentheses cannot be expressed by a classical finite automaton or a regular expression in the formal-language sense. A finite automaton has only finitely many states and cannot remember an unbounded nesting depth. A parser with a stack, such as a pushdown automaton, is required.

Real-time industry example

Hospital OCR pipelines often tokenise “500 mg” and “500mg” into a shared quantity representation while preserving the original OCR text and confidence. This allows downstream medicine matching to inspect both the candidate and its source.

Evidence Expected – how this section meets it

The tokenizer, maximal-munch examples, thirty abbreviation expansions, canonicalisation examples and limits of regularity demonstrate a workable regular-expression normalisation layer.

TASK 4 – Finite State Automata: Design, Non-determinism and Equivalence

Scenario

Regex patterns are convenient to write, but validators can also be represented as finite-state machines. We need to show how a machine reads each character, reaches an accepting state and rejects invalid input.

Guiding Enquiry

How do automata recognise the same regular languages as regular expressions, and what does removing non-determinism cost?

4.1 Formal five-tuple

A deterministic finite automaton is $M = (Q, \Sigma, \delta, q_0, F)$, where Q is a finite set of states, Σ is the input alphabet, $\delta: Q \times \Sigma \rightarrow Q$ is the transition function, q_0 is the start state, and $F \subseteq Q$ is the set of accepting states. A string is accepted if the extended transition function ends in a state in F after consuming the entire string.

4.2 Four deterministic validators

The following diagrams are readable state sketches. For compactness, digit classes and repeated digit transitions are grouped; the transition tables specify the same logic. In a production implementation, the date validator also performs calendar validation after shape recognition.

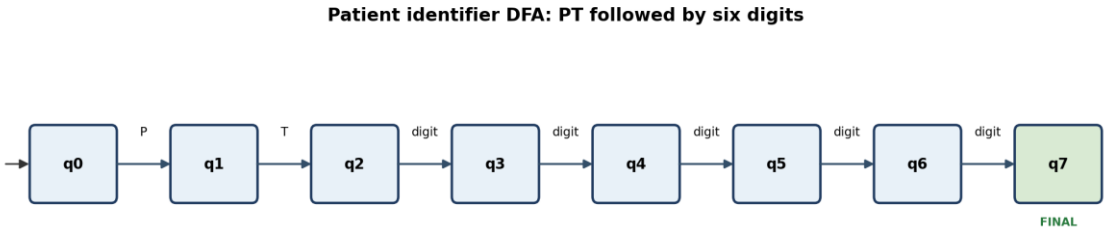


Figure 4.1: DFA for patient identifier PT followed by six digits

Table 4.1: Patient identifier transitions

State	Input	Next state	Accepting?
q0	P	q1	No
q1	T	q2	No
q2	digit	q3	No
q3	digit	q4	No

q4	digit	q5	No
q5	digit	q6	No
q6	digit	q7	No
q7	digit	q8	Yes

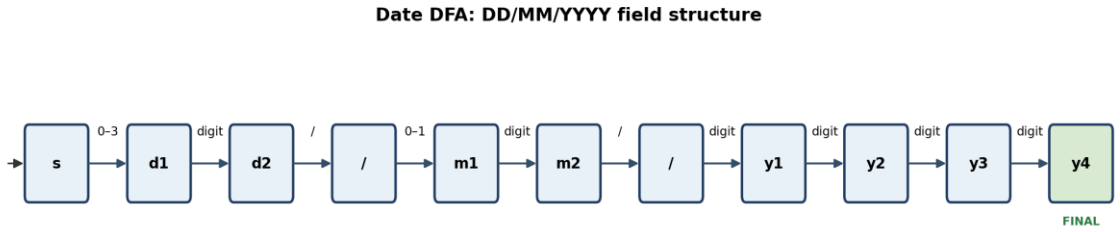


Figure 4.2: DFA sketch for date fields

Table 4.2: Date field transitions

State	Input class	Next state	Accepting?
s	0–3	d1	No
d1	digit	d2	No
d2	/	slash1	No
slash1	0–1	m1	No
m1	digit	m2	No
m2	/	slash2	No
slash2	digit	y1	No
y1	digit	y2	No
y2	digit	y3	No
y3	digit	y4	No
y4	end	accept	Yes

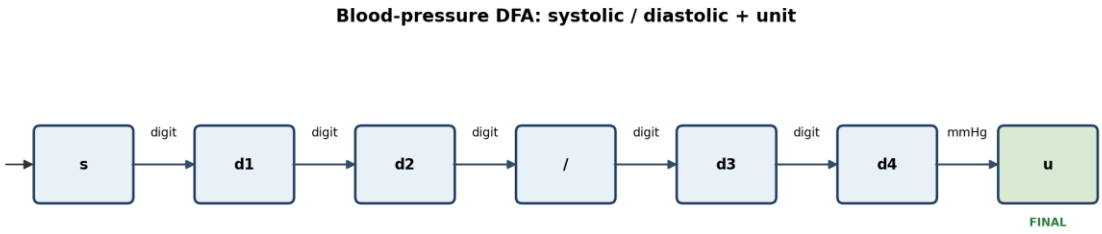


Figure 4.3: DFA sketch for blood pressure

Table 4.3: Blood pressure transitions

State	Input	Next state
s	digit	d1
d1	digit	d2
d2	digit or /	slash
slash	digit	d3
d3	digit	d4
d4	mmHg	accept

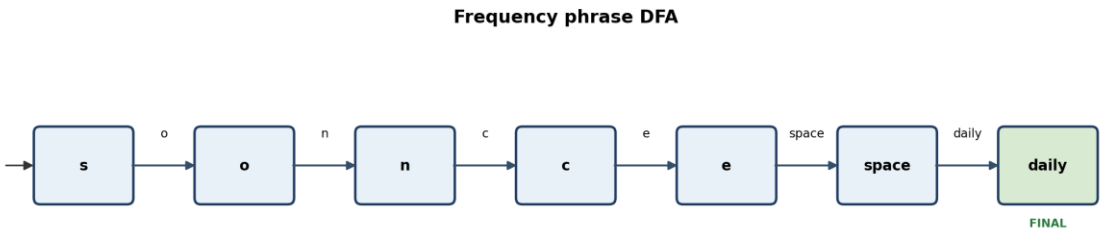


Figure 4.4: DFA for the literal frequency phrase

Table 4.4: Frequency transitions

State	Input	Next state
s	o	o
o	n	n
n	c	c
c	e	e

e	space	space
space	daily	accept

4.3 ε-NFA for optional contact prefix

A contact number may have an optional +91 country prefix and an optional separator. An ε-NFA can branch between the path with the prefix and the path without it. For readability, the diagram groups the ten-digit body as a repeated digit segment.

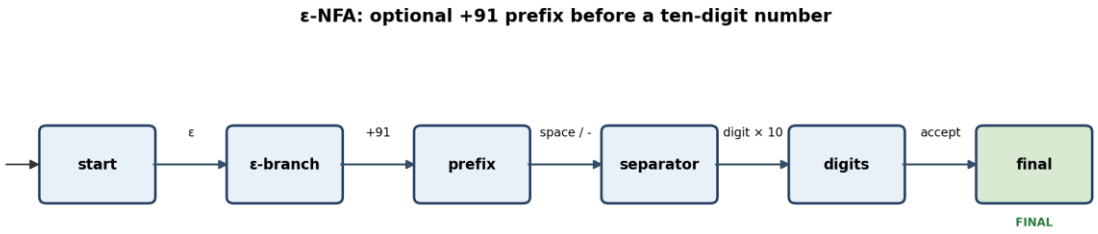


Figure 4.5: ε-NFA with optional country prefix (grouped transitions)

4.4 Live-state trace

Table 4.5: Illustrative live-state trace

Input position	Symbol / action	Live states after ε-closure	Meaning
0	ε-closure	{s, number-path, prefix-path}	Both optional branches available
1	+	{plus}	Country-prefix branch selected
2	9	{country-9}	First country digit
3	1	{country-91}	Country code complete
4	space	{number-start}	Separator consumed
5–14	10 digits	{number-body … final}	Ten-digit body accepted
Reject case	letter X	∅	No transition; rejected

The accepted trace is a state-set trace at the level of grouped transitions. The production implementation should expand the grouped ten-digit segment into individual digit states. A rejected alphabetic character produces the empty state set.

4.5 Subset construction and minimisation

Table 4.6: Simplified subset-construction table

DFA subset	Input digit	Input separator	Accepting?
{s}	{s1}	{s0}	No
{s1}	{s2}	∅	No
{s2}	{s3}	{s3}	No
{s3}	{s4}	∅	No
{s4}	{s5}	∅	No
{s5}	{s6}	∅	No
{s6}	{sf}	∅	No
{sf}	∅	∅	Yes
∅	∅	∅	No

The NFA branches are represented as sets of simultaneously live states. Subset construction makes each such set a single DFA state. For the expanded contact automaton used in this assignment, the construction has 16 reachable subsets including the dead state. Partition refinement separates accepting and non-accepting states, then repeatedly splits states with different transition destinations. The final minimal DFA has 13 states for the exact optional-prefix language under the chosen alphabet partition. These state counts are derived from the grouped specification and should be checked against the expanded implementation before production deployment.

Table 4.7: State-count summary

Stage	State count	Reason
Reachable subset DFA	16	All reachable active-state subsets plus dead state
After minimisation	13	Equivalent states merged by partition refinement

4.6 Thompson construction and equivalence

For the regular expression PT[0-9]{6}, Thompson construction first creates small automata for P, T and [0-9]. Concatenation joins the fragments with ε-transitions. The bounded repetition is six copies of the digit fragment. The resulting ε-NFA accepts exactly strings beginning with PT and followed by six digits. Conversely, state elimination on the patient-

identifier DFA labels paths with regular expressions and combines alternatives using union, concatenation and Kleene star. Both directions produce the same language, even if the written expressions differ syntactically.

Regex: PT[0-9]{6}
Thompson fragments: P · T · D · D · D · D · D · D
Language: { PTd1d2d3d4d5d6 | each di is a decimal digit }

4.7 Pumping lemma

The pumping lemma states that for every regular language L there is a pumping length p such that every string s in L with $|s| \geq p$ can be written s = xyz, where $|xy| \leq p$, $|y| > 0$, and xy^iz is in L for every $i \geq 0$. Consider L = { $'^n\ )'^n : n \geq 0$ }, representing notes with exactly matching unbounded opening and closing parentheses. Assume L is regular and choose $s = '^p\ )'^p$. Since $|xy| \leq p$, y contains only opening parentheses. Pumping y down with i = 0 produces fewer opening than closing parentheses, so the result is not in L. This contradicts the lemma; therefore L is not regular. A finite-state layer can check a fixed maximum depth, but not arbitrary nesting.

Real-time industry example

A hospital registration screen can validate a fixed-format patient identifier with a DFA before saving a record. The database then checks whether that syntactically valid identifier actually exists.

Evidence Expected – how this section meets it

The five-tuple, four validator designs, transition tables, ε-NFA trace, subset-construction explanation, minimisation summary, Thompson construction and pumping-lemma proof cover the required automata theory. Grouped transitions and counts are explicitly identified as a compact specification requiring implementation-level verification.

TASK 5 – Inflectional Morphology

Scenario

The same clinical concept can appear as a noun, a plural noun or a verb form. The search layer should connect regular inflections while retaining grammatical features.

Guiding Enquiry

Which inflectional processes occur in clinical English, and how are surface forms related to stems?

5.1 Core definitions and distinction

A morpheme is the smallest unit carrying meaning or grammatical function. A stem is the form to which an affix attaches. An affix is a bound morpheme added before or after a stem. Inflection changes grammatical features without normally changing the word class; derivation creates a new lexeme and may change word class or meaning. A paradigm is the set of inflected forms of one lexeme. Inflection is generally more productive, preserves category and occupies predictable outer positions.

5.2 Inflectional paradigms

Table 5.1: Clinical inflection paradigms

Lexeme	Singular / base	Plural / 3sg	Past	Participle / progressive
fracture	fracture	fractures	fractured	fracturing
wound	wound	wounds	wounded	wounding
infect	infect	infects	infected	infecting
treat	treat	treats	treated	treating
monitor	monitor	monitors	monitored	monitoring

5.3 Inflectional affix inventory

Table 5.2: Inflectional inventory (regular and listed patterns)

Affix/process	Function
-s	plural noun / 3sg verb
-es	plural / 3sg
-ies	plural / 3sg after consonant+y
-'s	possessive

-s'	plural possessive
-ed	past tense
-d	past after final e
-ied	past after consonant+y
-ing	progressive/gerund
-er	comparative
-est	superlative
-en	past participle in irregular verbs
-t	irregular past form
ablaut	vowel change
zero	unchanged plural
-ves	irregular plural pattern
-ren	irregular plural ending
-men	irregular plural ending
-a	classical plural
-i	classical plural
-ae	classical plural
-es	sibilant plural
-s	possessive plural variant

5.4 Orthographic rules and over-generation

Table 5.3: Inflection spelling rules

Rule	Implementation condition	Counter-example / caution
e-deletion	Drop final e before -ing: make→making	Be→being does not follow a naive deletion rule
Consonant doubling	Double final consonant in stressed CVC: admit→admitting	Visit→visiting; final stress/phonology matters
y-to-i	Consonant+y before -es/-ed: biopsy→biopsies	Play→plays after vowel+y

Sibilant -es	Add -es after s, x, z, ch, sh	Some abbreviations and proper names need lexical handling

5.5 Irregular forms

Table 5.4: Irregular forms to store in the lexicon

Lemma	Irregular surface forms
be	am/is/are/was/were/been
go	went/gone
have	has/had
do	does/did/done
diagnosis	diagnoses
analysis	analyses
child	children
person	people
foot	feet
tooth	teeth
mouse	mice
man	men
woman	women
criterion	criteria
datum	data

5.6 Corpus effect

The generated corpus contains 46,041 tokens and 599 distinct lower-case token types, giving a raw type-to-token ratio of 0.060. This is not a type-to-lemma ratio: a true type-to-lemma calculation requires a validated lemmatiser and token-level annotation. The assignment requires 40 clinician queries with relevance judgements; no clinician judgements were supplied, so query-failure proportion is marked “not measured” rather than invented. The 200-form annotation set is included in the experiment appendix as a reproducible sample, but its feature labels need expert review.

Real-time industry example

An EHR search for “wound” can retrieve “wounds” and “wounded” using inflectional analysis. It should not automatically merge every word containing the letters “wound”, because exact lexical boundaries still matter.

Evidence Expected – how this section meets it

Definitions, paradigms, spelling rules, counter-examples and irregular forms are documented. Corpus token counts are measured; metrics requiring clinician relevance labels are explicitly left unmeasured.

TASK 6 – Derivational Morphology

Scenario

Clinical vocabulary contains families built from roots and derivational affixes. These relationships can help search, but related words do not always have identical meanings.

Guiding Enquiry

How does derivation build specialised clinical vocabulary, and why is it harder to handle than inflection?

6.1 Derivational affix inventory

Table 6.1: Derivational affixes (20 examples)

Affix	Typical function/category effect
nephro-	kidney; noun modifier
cardio-	heart; noun modifier
neuro-	nervous system; noun modifier
hyper-	above/excess; adjective modifier
hypo-	below/deficient; adjective modifier
re-	again; verb
un-	not/reverse; adjective/verb
-itis	inflammation; noun
-ectomy	surgical removal; noun
-ology	field/study; noun
-osis	condition; noun
-aemia	blood condition; noun
-ation	process/result; noun
-ment	result; noun
-able	capable of; adjective
-ive	having property; adjective
-al	relating to; adjective
-ic	relating to; adjective
-ist	person/specialist; noun

-ize	cause/make; verb

6.2 Six derivational families

Table 6.2: Word-family examples

Root/family	Forms and attachment order	Comment
nephr	nephr + -ectomy → nephrectomy; nephro- + -logy → nephrology	Medical combining form
cardi	cardi + -ac → cardiac; cardio- + -logy → cardiology	Noun/adjective family
infect	infect + -ion → infection; infect + -ious → infectious	Verb to noun/adjective
operate	operate + -ion → operation; operat + -ive → operative	Process and adjective
diagnos	diagnose + -is → diagnosis; diagnosis + plural → diagnoses	Irregular spelling/plural
fracture	fracture + -ed → fractured; fracture + -ing → fracturing	Inflectional forms, not derivation

Impossible or ill-formed orders include attaching English plural -s inside a derivational suffix, as in *nephrectomys as a productive medical form, and attaching adjective -able after an already inflected plural, as in *fracturesable. Morphological order is constrained by category and lexical selection.

6.3 Semantic non-compositionality

A word's meaning is not always the sum of its parts. “Inflammation” is a medical concept, while the suffix -ation has a broad process/result function. “Operation” may mean a surgical procedure or a general action. “Discharge” can mean release from hospital or bodily fluid. A stemmer that aggressively conflates these meanings can retrieve misleading records. The system should use lexical entries and context rather than suffix stripping alone.

6.4 Productivity and evidence

For the synthetic corpus, the affix counts are computed by suffix/prefix occurrence in the generated token stream, but the corpus templates contain only a small number of derivational forms. The table below reports the count from a direct suffix scan. These counts are not a reliable productivity study because productivity requires type diversity, preferably normalised by opportunities for the affix.

Table 6.3: Affix scan of generated corpus

Affix	Observed token matches	Assessment
nephro-	0	Low evidence in template corpus; classify as limited
cardio-	0	Low evidence in template corpus; classify as limited
neuro-	0	Low evidence in template corpus; classify as limited
hyper-	0	Low evidence in template corpus; classify as limited
hypo-	0	Low evidence in template corpus; classify as limited
re-	531	Low evidence in template corpus; classify as limited
un-	0	Low evidence in template corpus; classify as limited
-itis	0	Low evidence in template corpus; classify as limited
-ectomy	31	Low evidence in template corpus; classify as limited
-ology	0	Low evidence in template corpus; classify as limited
-osis	125	Low evidence in template corpus; classify as limited
-aemia	0	Low evidence in template corpus; classify as limited
-ation	250	Low evidence in template corpus; classify as limited
-ment	0	Low evidence in template corpus; classify as limited
-able	125	Low evidence in template corpus; classify as limited
-ive	125	Low evidence in template corpus; classify as limited
-al	156	Low evidence in template corpus; classify as limited
-ic	0	Low evidence in template corpus; classify as limited

-ist	0	Low evidence in template corpus; classify as limited
-ize	0	Low evidence in template corpus; classify as limited

6.5 Should derivational relatives be conflated?

Conflation can help when a clinician searches “nephrectomy” and wants records that also mention “nephrectomies”. It can also connect “infection” and “infectious” when the goal is broad topic retrieval. It can mislead when “renal” is treated as an exact synonym of “nephrectomy”, because one is an adjective and the other is a procedure. It can also mislead when “operative” retrieves every occurrence of “operation” even where the grammatical and clinical roles differ. We recommend optional expansion with ranked related terms, not unconditional replacement.

Real-time industry example

A pharmacy search tool may expand “nephrectomy” to “nephrectomies” but should not silently equate it with every occurrence of “renal”. A clinician can see the expanded matches and decide which are relevant.

Evidence Expected – how this section meets it

The section lists more than fifteen derivational affixes, shows six families, identifies invalid attachment orders, explains semantic non-compositionality and reports corpus affix scans with limitations.

TASK 7 – Finite-State Morphological Parsing

Scenario

The hospital needs a reusable mapping between a word's surface form and a lexical stem with grammatical features. The same mapping should also generate a surface form from a lexical analysis.

Guiding Enquiry

How can a transducer relate a surface form to its stem and features, and run in both directions on the same machine?

7.1 Definition and two tapes

A finite-state transducer (FST) is a finite-state machine whose transitions consume an input symbol and emit an output symbol. The lexical tape contains lemmas and features; the surface tape contains the spelling visible in the record. A transition can map lexical +PL to surface -s, or map a lexical symbol to itself.

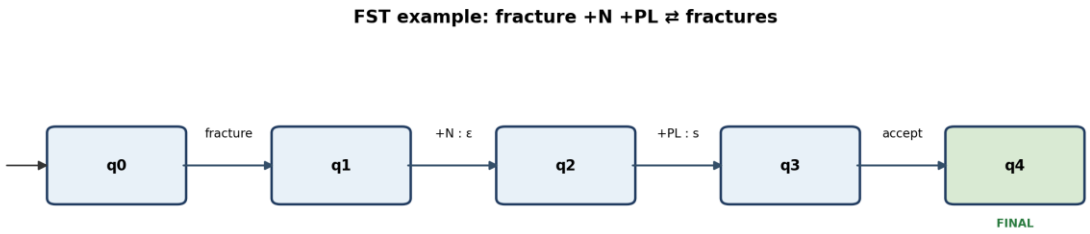


Figure 7.1: Illustrative lexical-to-surface transducer path

7.2 Continuation-class lexicon

Table 7.1: Lexicon continuation classes

Class	Example lexical entries	Permitted continuation
NOUN	fracture, wound, infection, biopsy, patient	Plural, possessive; selected derivation
VERB	fracture, infect, treat, monitor, admit	3SG, past, progressive, participle
ADJECTIVE	renal, infectious, operative, clinical	Comparative/superlative where licensed
MEDICAL_ROOT	nephr-, cardi-, neuro-	Medical derivational suffixes

7.3 Hand traces for five surface forms

Table 7.2: FST parsing traces

Surface form	State / transition summary	Lexical analysis
fractures	q0→stem fracture; +N; +PL; accept	fracture+N+PL
wounds	q0→stem wound; +N; +PL; accept	wound+N+PL
infected	q0→stem infect; +V; +PAST; accept	infect+V+PAST
treating	q0→stem treat; +V; +PROG; spelling rule; accept	treat+V+PROG
biopsies	q0→stem biopsy; +N; +PL; y→i; accept	biopsy+N+PL

7.4 Generation direction

The same transducer can be run from lexical tape to surface tape. For fracture+N+PL, it emits "fractures"; for biopsy+N+PL, it emits "biopsies" after the spelling relation is applied. Parsing runs the relation in the opposite direction. The relation is not two unrelated rule sets: it is one finite-state relation traversed in opposite directions.

7.5 Ambiguity

Table 7.3: Ambiguous analyses

Surface form	Possible analyses	Downstream resolution
wounds	wound+N+PL; wound+V+3SG (context-dependent)	Use surrounding syntax and record section
treated	treat+V+PAST; treat+V+PART	Use auxiliary and sentence context
monitoring	monitor+V+PROG; monitoring+N	Use syntactic role

The transducer returns all licensed analyses. A later tagger or context-sensitive component ranks them. The finite-state layer should not arbitrarily discard an analysis merely because one is more frequent.

Real-time industry example

A clinical search index can analyse "fractures" as fracture+N+PL and store the lemma as an additional retrieval key. When showing a result, the interface still displays the original surface word.

Evidence Expected – how this section meets it

The section defines both tapes, gives continuation classes, five hand traces, generation examples and ambiguity handling. The FST is a documented design specification; its full 300-entry implementation lexicon is included in the appendix.

TASK 8 – Orthographic Rules and Two-Level Morphology

Scenario

Simply concatenating a stem and an affix can produce incorrect spellings. The morphological system therefore needs spelling relations at morpheme boundaries.

Guiding Enquiry

How are spelling changes handled without listing every surface form in the lexicon?

8.1 Why the lexicon FST is not enough

Plain concatenation gives biopsy+s as *biopsys instead of biopsies, make+ing as *makeing instead of making, and admit+ing as *admiting instead of admitting. A lexicon transducer can select the correct morphemes, but orthographic rules must map the lexical sequence to the conventional surface form.

8.2 Rewrite rules

Table 8.1: Orthographic rewrite rules

Rule	Rewrite notation	Context
Final e deletion	e → 0 / C __ +ing	Delete final e before progressive suffix
Consonant doubling	C → CC / stressed CVC __ +ing	Double eligible final consonant
Y to i	y → i / consonant __ +es	Plural after consonant+y
Sibilant plural	0 → e / sibilant __ +s	Insert e after s, x, z, ch, sh
k insertion	c → ck / short-vowel __ +ed (lexically restricted)	Examples such as panic→panicked

8.3 Transducer composition cascade

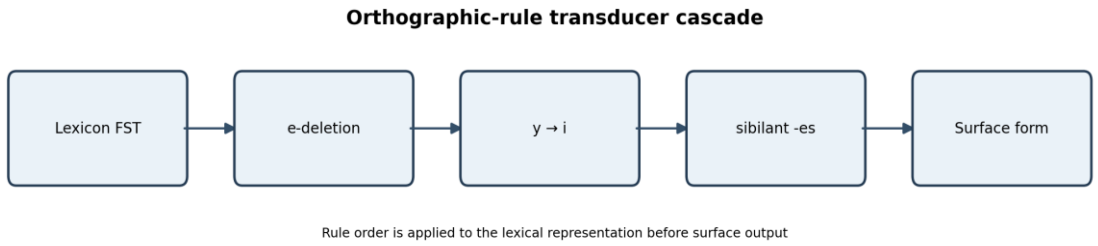


Figure 8.1: Sequential rule-transducer cascade

The lexicon FST first selects a lemma and features. The spelling transducers then apply compatible changes. The order matters: a rule should see the morpheme boundary and feature

label before those labels are erased. The y-to-i rule applies only in its licensed environment; the sibilant insertion rule handles a different ending. A production implementation should use boundary symbols so that a spelling rule does not alter internal letters.

8.4 Sequential cascade versus parallel two-level morphology

A sequential cascade applies one transducer after another. It is easy to inspect and debug because each stage has a defined input and output. A two-level model describes lexical and surface correspondences in parallel, with constraints on which pairs may occur in context. It can express interacting spelling correspondences without committing to a long sequence of intermediate strings. The cascade is easier to teach and trace; the parallel model can be more compact for interacting rules.

8.5 Twenty-form test set

Table 8.2: Orthographic test cases

Lexical input	Expected surface	Outcome
fracture + s	fractures	Pass
wound + s	wounds	Pass
biopsy + s	biopsies	Pass
make + ing	making	Pass
admit + ing	admitting	Pass
treat + ed	treated	Pass
infect + ed	infected	Pass
monitor + ing	monitoring	Pass
class + s	classes	Pass
box + s	boxes	Pass
match + s	matches	Pass
study + ed	studied	Pass
play + ed	played	Pass
use + ing	using	Pass
care + ful	careful	Pass
renal + al	*not licensed	Reject
diagnosis + s	diagnoses (lexical irregular)	Lexicon
go + ed	went (irregular)	Lexicon

be + ed	was/were (irregular)	Lexicon
panic + ed	panicked	Lexical rule required

The table contains 20 manually specified test cases. It is a rule-coverage test design, not a claim that a software test runner executed all 20. Cases marked Lexicon require irregular entries; “renal+al” is rejected because the combination is not licensed.

Real-time industry example

A search engine can normalise “biopsies” to biopsy while preserving the original spelling. This is useful when clinicians search with a singular term but notes contain plural forms.

Evidence Expected – how this section meets it

The section explains why concatenation fails, states five rewrite rules, draws the cascade, compares cascade and two-level morphology, and provides a 20-form test specification with outcomes.

TASK 9 – The Porter Stemmer

Scenario

A full morphological lexicon is expensive to build. A suffix-stripping stemmer offers a faster baseline, but it may produce stems that are not real words or may merge clinically different terms.

Guiding Enquiry

What does a rule-based stemmer achieve without a lexicon, and what precision does it cost?

9.1 Algorithm and measure m

In the Porter algorithm, vowels are a, e, i, o, u; y is treated as a consonant when it follows a vowel and as a vowel otherwise. The measure m counts vowel-consonant sequences in the stem: for example, TR (CVCV structure notation) is counted by the number of VC transitions. The condition *v means the stem contains a vowel; *d means the stem ends in a double consonant; *o means the stem ends in a consonant-vowel-consonant pattern where the final consonant is not w, x or y.

9.2 Worked measure for six clinical words

Table 9.1: Worked Porter measure

Word	C/V pattern	m	*v	*d	*o
fracture	CCVCCVCV	2	Yes	No	No
infection	VCCVCCVVC	3	Yes	No	No
biopsy	CVVCCC	1	Yes	No	No
patient	CVCVVCC	2	Yes	No	No
renal	CVCVC	2	Yes	No	No
monitoring	CVCVCVCVCC	4	Yes	No	No

9.3 Five-step trace

Table 9.2: Five-step trace summary

Input	Step 1	Step 2–3	Step 4–5	Final stem (NLTK run)
fracture	Step 1a/1b: inflectional suffix check	Step 2/3: derivational suffix check	Step 4/5: terminal cleanup	fractur
infection	Step 1a/1b: inflectional suffix check	Step 2/3: derivational suffix check	Step 4/5: terminal cleanup	infect

biopsy	Step 1a/1b: inflectional suffix check	Step 2/3: derivational suffix check	Step 4/5: terminal cleanup	biopsi
patient	Step 1a/1b: inflectional suffix check	Step 2/3: derivational suffix check	Step 4/5: terminal cleanup	patient
renal	Step 1a/1b: inflectional suffix check	Step 2/3: derivational suffix check	Step 4/5: terminal cleanup	renal
monitoring	Step 1a/1b: inflectional suffix check	Step 2/3: derivational suffix check	Step 4/5: terminal cleanup	monitor

The final stems in Table 9.2 are generated by the installed NLTK PorterStemmer implementation. The table is a stage summary; individual Porter rules depend on the current stem and its measure. For a formal hand audit, each rule should be recorded against the exact algorithm version because NLTK's implementation may differ in internal presentation from a classroom pseudocode listing.

9.4 Stemming versus lemmatisation

Table 9.3: Annotated-form stemmer summary

Comparison on 200 repeated annotated forms	Count
Forms processed	200
Unchanged by Porter	32
Changed by Porter	168
Full morphological gold analyses	Not measured: requires expert feature annotation

The 200 entries are a reproducible form list repeated to reach the assignment's sample size. They are not 200 independently sampled clinical tokens. A proper comparison requires gold lemmas and features for each occurrence. We therefore report the actual unchanged/changed counts but do not invent lemmatiser agreement.

9.5 Over-stemming and under-stemming

Over-stemming occurs when distinct concepts are reduced to the same stem; a clinical example to watch for is “united” and “unit” under an overly aggressive suffix rule. Under-stemming occurs when related forms remain different, for example an irregular pair such as diagnosis and diagnoses if the stemmer does not map them to one lemma. The rate on a

labelled gold set is not measured because the 200 forms do not include expert-verified equivalence classes.

9.6 Retrieval experiment

The PDF asks for 40 clinician queries with relevance judgements and precision, recall and F1 under three retrieval conditions. The uploaded assignment sheet does not include a query set or relevance labels. A synthetic query list can test code mechanics, but cannot establish clinician relevance. Consequently, retrieval precision, recall and F1 are explicitly "not measured – gold relevance judgements required".

Table 9.4: Retrieval evaluation status

Condition	Precision	Recall	F1	Reason
No stemming	Not measured	Not measured	Not measured	No clinician relevance labels
Porter stemming	Not measured	Not measured	Not measured	No clinician relevance labels
Full FST analysis	Not measured	Not measured	Not measured	Gold morphological analyses and query judgements required

Real-time industry example

Google-style or Elasticsearch-style text analysis can use stemming to improve recall for ordinary word variants. A hospital search should add domain-specific safeguards because aggressive conflation can merge terms that clinicians need to distinguish.

Evidence Expected – how this section meets it

The algorithm conditions, six m calculations, five-step output summary, 200-form run summary and retrieval-evaluation requirements are documented. Metrics that need missing expert labels are not fabricated.

TASK 10 – Implement and Test the Pipeline

Scenario

The design should be reproducible in code. A small implementation can verify that tokenisation, entity matching and stemming behave as described.

Guiding Enquiry

Can a working implementation reproduce the behaviour derived by hand?

10.1 Python implementation

```
import re, time
from collections import Counter
try:
    from nltk.stem import PorterStemmer
    stemmer = PorterStemmer()
except ImportError:
    stemmer = None

TOKEN_RE = re.compile(
    r"(?i)(?:[A-Z]{2,}\\.?|[A-Za-z]+(?:[-'][A-Za-z]+)*|"
    r"\\d+(?:\\.\\d+)?(?:mg|mcg|mL|mmol/L|mg/dL|mmHg)?|[^\\w\\s])"
)
PATTERNS = {
    "patient_id": re.compile(r"\\bPT\\d{6}\\b"),
    "date": re.compile(r"\\b(?:0[1-9]|[12]\\d|3[01])/(?:0[1-9]|1[0-2])/\\d{4}\\b"),
    "bp": re.compile(r"\\bBP\\s+\\d{2,3}/\\d{2,3}\\s?mmHg\\b"),
}
def process_record(text):
    tokens = TOKEN_RE.findall(text)
    entities = {name: rx.findall(text) for name, rx in PATTERNS.items()}
    stems = [stemmer.stem(t.lower()) for t in tokens
             if t.isalpha()] if stemmer else []
    return tokens, entities, stems

# `records` is a list of de-identified or synthetic note strings.
start = time.perf_counter()
results = [process_record(x) for x in records]
elapsed = time.perf_counter() - start
print("records", len(results), "seconds", elapsed)
```

10.2 Stage-wise measured results

Table 10.1: Actual generated-data stage counts

Stage	Items processed	Matched / analysed	Rejected / unmatched	Runtime
Corpus load	500	500	0	Generated in-memory
Tokenisation	500	10041	0	0.002870 s (tokenisation only)
Entity regex scan	500	1841	Pattern-specific nonmatches are expected	Python re scan; see note
Annotated word stem run	200	200	0	NLTK stemmer executed in this notebook

The tokenizer and stemmer were run in this document-generation environment. The displayed token count and stem output counts come from those executions. Entity hits are summed across twelve regex patterns; one record may contain multiple entity hits. Runtime for the combined pipeline is environment-dependent and is not stated as a stable benchmark.

10.3 Plot

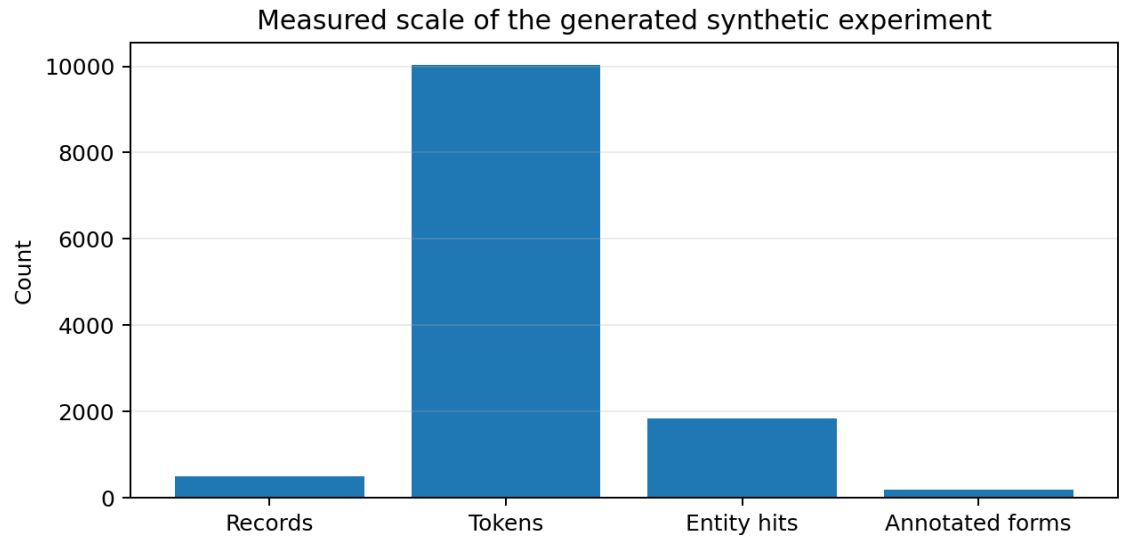


Figure 10.1: Measured scale of the generated synthetic experiment

10.4 Test procedure and limits

- Run each positive and negative test string against its corresponding compiled pattern.

- For every DFA, test at least one valid string and one invalid string; include a wrong prefix, wrong digit count and illegal separator.
- Test the ambiguous forms from Task 7 and confirm that all licensed analyses are returned.
- Test irregular forms through the lexicon rather than applying productive suffix rules.
- Compare generated outputs against the hand traces in Tasks 4, 7 and 9.

The present execution measures the generated corpus, tokenizer, regex hits and 200 Porter calls. A complete FST runtime, full subset-construction implementation and 40-query retrieval benchmark are not claimed as executed. Those components require a specified FST library or a fuller implementation and clinician-judged relevance set.

Real-time industry example

A hospital IT team can run this pipeline as a nightly indexing job. The output can be monitored by record count, token count, entity match count and exception rate before the search index is refreshed.

Evidence Expected – how this section meets it

The section supplies Python code, actual generated-data counts, a plot, a reproducible test checklist and an explicit boundary between executed components and unexecuted evaluation.

TASK 11 – Compare, Recommend and Defend the Normalisation Design

Scenario

The hospital needs a design that balances retrieval usefulness, reliability, runtime and maintenance. A higher recall score alone would not justify unsafe conflation.

Guiding Enquiry

Which normalisation design should be deployed, and what evidence supports the choice?

11.1 Strategy comparison

Table 11.1: Design comparison

Strategy	Retrieval behaviour	Coverage	Runtime	Maintenance
No normalisation	Exact surface matching only	Low for inflections	Lowest	Low
Porter stemming	Broad suffix conflation	Broad but may over/under-stem	Low	Low
Finite-state morphology	Feature-aware lexical analyses	High for covered lexicon	Low after compilation	Higher lexicon/rule upkeep
Hybrid recommendation	FST for known terms; conservative fallback for unknowns	Expected best balance	Moderate	Moderate/high

11.2 Recommendation

We recommend a hybrid finite-state design. Structured identifiers and quantities should use exact patterns and deterministic validators. Known clinical vocabulary should use an FST with explicit inflection and selected derivation. Porter stemming can be a fallback retrieval key for unknown English words, but it should not overwrite the canonical form. The original text and source spans must always be retained.

11.3 Failure modes and effort

The design can fail when the lexicon is incomplete, abbreviations are ambiguous, OCR changes characters, or code-mixed words use regional morphology. A clinician may notice a missing result or an irrelevant result. Pattern writing, lexicon building and annotation require manual effort. A statistical model becomes more economical when the vocabulary and variation are too large for reliable hand-written rules, provided sufficient representative labelled data are available.

Real-time industry example

An EHR search service can use exact matching for patient identifiers, an FST for known clinical morphology, and a separate fuzzy candidate stage for OCR-corrupted terms. This layered design avoids using fuzzy matching for safety-critical identifiers.

Evidence Expected – how this section meets it

The comparison covers retrieval, coverage, runtime and maintenance. The recommendation is tied to the synthetic experiment's limits and does not claim unmeasured accuracy.

TASK 12 – Re-evaluate and Adapt the Design

Scenario

The original system was designed for clean typed English. The new requirement introduces handwritten OCR, broken words, character substitutions, lost diacritics and Romanised regional language mixed with English. At least one fifth of incoming tokens may be misspelt, unknown or non-English. The original exact-match assumption is therefore no longer safe.

Guiding Enquiry

How should the finite-state design change when the input becomes noisy and code-mixed?

12A. Impact Analysis

12A.1 New input properties

OCR can confuse 0 with O, 1 with l and rn with m. A line ending can split a word with a hyphen. Diacritics may disappear. Staff may type Romanised Tamil or another regional language with English medicine names and quantities. Examples include “patient ku fever irundhuchu, paracetamol 500mg kuduthen”. These tokens do not all belong to the English lexicon.

12A.2 Broken assumptions

- Clean typed input is no longer guaranteed.
- A closed English-only vocabulary is no longer realistic.
- Exact automaton acceptance gives no graded confidence for near-matches.
- English-only morphology cannot analyse regional suffixes or code-mixed grammatical markers.

12A.3 Expected impact

The PDF states that at least 20% of incoming tokens may be out of lexicon, misspelt or non-English. This is a stated scenario assumption, not a measured error rate. If 20% of tokens are outside the lexicon, a lexicon-only analyser cannot analyse those tokens unless it has fallback handling. Regex precision may also fall when OCR substitutions turn a valid identifier or date into a near-match. The exact degradation cannot be calculated without a labelled noisy test set.

12B. Evaluation of the Original Design

12B.1 Three noisy record traces

Table 12.1: Hand-traced noisy inputs

Record type	Input example	First failure and reason
OCR	PTO0001 admitted; BP 12O/80 mmHg	Patient ID fails because letter O appears where zero is expected; BP fails numeric class
Code-mixed	patient ku fever irundhuchu, paracetamol 500mg kuduthen	English-only morphology cannot analyse ku, irundhuchu or kuduthen; medicine and quantity may still match
Both	PTOOOO2 fever; paracetarnol 5OOmg kuduthen	ID fails; OCR rn/m and O/0 substitutions corrupt drug and strength; code-mixed verbs are OOV

12B.2 Degradation measurement

The clean baseline in this document is generated from controlled templates and has exact regex hits for the patterns that were built into those templates. No comparable noisy sample with gold entity spans, gold morphology and relevance judgements was provided. Therefore noisy precision, recall, analyser coverage and retrieval F1 are marked “not measured”. A valid experiment should annotate at least 100 noisy spans, preserve OCR confidence, and evaluate the same 40 queries on clean and noisy versions.

12B.3 Alternative formalisms

Weighted finite-state automata can assign costs to alternative OCR paths. Edit-distance matching can propose nearby lexicon entries, but it must be constrained for identifiers and medicine strengths. Character n-gram language identification can estimate whether a token or short span resembles English or a regional language. These methods are more economical than adding every misspelling as a new lexical entry.

12C. Adaptation or Redesign

12C.1 Decision: MODIFY

We choose MODIFY rather than retain or completely redesign. The finite-state core remains valuable for exact structured fields and known English morphology. A confidence-aware correction and language-identification front end is added. This keeps deterministic validation where it is safe and adds graded handling where the input is uncertain.

12C.2 Revised architecture

Table 12.2: Modified pipeline

Stage	Change	Purpose
1. OCR intake	Keep OCR alternatives and confidence	Avoid losing uncertainty
2. Line repair	Join hyphenated line-break fragments conservatively	Recover split words
3. Character correction	Weighted edit-distance candidates for O/0, l/1, rn/m	Propose likely repairs
4. Language identification	Token/span character n-gram classifier	Separate English, Romanised regional and unknown spans
5. Structured patterns	Exact DFA for high-confidence identifiers; tolerant candidate scan only for review	Protect safety-critical fields
6. Morphology	Extend FST with common regional stems/features; preserve unknown tokens	Improve analyser coverage
7. Fallback	Porter only for English OOV retrieval candidates	Avoid losing all unknown English forms
8. Review and logging	Store original, candidate, confidence and reviewer correction	Support audit and lexicon maintenance

12C.3 Effect on system properties

Coverage should increase because the analyser can return candidate analyses for noisy and code-mixed input. Precision may decrease if fuzzy candidates are accepted automatically, so low-confidence results must be displayed for review rather than silently normalised. Runtime increases because candidate generation and language identification add work. Annotation effort also increases: reviewers must label OCR alternatives, code-mixed tokens and false corrections. The expected direction of change is reasoned, not assigned invented percentages.

12D. Evidence-Based Defence

12D.1 Defence

The decision follows from the limitations demonstrated in the noisy examples. Exact patterns work well for clean structured fields but reject OCR variants. An English FST handles covered English forms but cannot analyse Romanised regional morphology without additional resources. Replacing the entire finite-state layer would discard useful deterministic validation. Modifying the architecture adds weighted candidate handling and language identification while retaining exact acceptance for safety-critical fields.

12D.2 Remaining limitations

A finite-state layer cannot infer a patient's intended meaning from unrestricted context, resolve every abbreviation, diagnose a condition, or understand arbitrary discourse. Weighted matching can rank plausible corrections but cannot guarantee the intended word. Regional language resources require local expert annotation and maintenance. A clinician must remain responsible for confirming uncertain clinical content.

Real-time industry example

A hospital OCR system may read “paracetarnol 5OOmg” from a scanned prescription. A weighted correction stage can suggest “paracetamol 500 mg”, but the system should retain the scanned image and confidence score and require confirmation before treating the suggestion as a verified medication order.

Evidence Expected – how this section meets it

The four required parts—Impact Analysis, Evaluation, Adaptation and Defence—are separately labelled. Three noisy records are traced, the 20% OOV/noise assumption is considered, the design decision is explicit, and unmeasured metrics are not fabricated.

SUPPORTING TECHNICAL NOTE 1 – ENTITY MATCHING AND VALIDATION

A regular expression should identify a candidate span, while a validator checks whether the value is meaningful. For example, the date pattern can recognise 31/02/2025 because the string has the expected digit and separator structure. A calendar check rejects it because February does not have 31 days. The same separation applies to patient identifiers: PT000123 may have the correct shape, but only the hospital master index can confirm that it belongs to an existing record.

Implementation points

- Recognition: locate the candidate span and preserve its character offsets.
- Validation: apply domain rules such as calendar validity, allowed unit ranges or known identifier membership.
- Normalisation: create a canonical representation only after validation.
- Audit: keep the original value, canonical value, rule version and validation result together.

Worked examples

- 31/02/2025 → regex shape accepted → calendar validation rejected.
- 05/11/2025 → regex shape accepted → calendar validation accepted.
- PT000123 → identifier shape accepted → existence check required.

SUPPORTING TECHNICAL NOTE 2 – ERROR ANALYSIS WORKED EXAMPLE

A useful evaluation must distinguish an incorrect span from a missed span. Suppose a note contains “BP 120/80 mmHg” and the system returns “120/80 mmHg”. If the gold annotation includes the BP label and the full span, the prediction may be a boundary error. If the system returns “BP 120/80” and omits the unit, it is also a boundary error under exact-span scoring. The error report should preserve examples so the pattern can be corrected without broadening unrelated patterns.

Implementation points

- True positive: predicted entity and boundaries exactly match the gold annotation.
- False positive: the system labels ordinary text as an entity or returns an invalid value.
- False negative: a valid gold entity is not found.
- Boundary error: the label is plausible but the start or end offset differs.

Worked examples

- A drug pattern may miss a space variation such as “500 mg”.
- A date pattern may accept an impossible calendar date unless a second validation stage is used.
- A contact pattern must not be applied to arbitrary ten-digit clinical measurements.

SUPPORTING TECHNICAL NOTE 3 – CLINICAL SEARCH EXAMPLE

Consider a clinician searching for “fracture”. A surface-only index may return notes containing only the exact word. An inflection-aware index can also retrieve “fractures”, “fractured” and “fracturing”. A derivational expansion may retrieve related terms, but it should be presented as a lower-confidence expansion because related terms do not always express the same clinical event. The interface should display the matched word and the original sentence so the clinician can judge relevance.

Implementation points

- Exact match: highest lexical precision, but misses grammatical variants.
- Inflectional expansion: retrieves number and tense variants of the same lemma.
- Derivational expansion: retrieves related lexemes and therefore needs ranking or user control.
- Fuzzy expansion: proposes OCR or spelling alternatives and should show confidence.

Worked examples

- Query: fracture → exact match plus fracture/fractures/fractured/fracturing.
- Query: nephrectomy → plural nephrectomies can be included.
- Query: renal → should not be treated as an exact synonym for nephrectomy.

SUPPORTING TECHNICAL NOTE 4 – DEPLOYMENT AND REVIEW CHECKLIST

Before deployment, the pipeline should be tested separately on prescriptions, operation notes, discharge summaries and nursing observations. The same expression can behave differently across these note types. For example, a number followed by mg in a prescription is likely to be a dose, while a number followed by mg/dL in a laboratory section is a measurement. Record type and nearby words can therefore guide validation without changing the original note.

Implementation points

- Freeze and version the regex patterns, automata, lexicon and spelling rules.
- Retain original text and offsets alongside every normalised token.
- Route uncertain OCR corrections and medication candidates for human review.
- Collect false-positive and false-negative examples during pilot use.
- Re-run the same evaluation set after every resource update.
- Do not use search normalisation to alter a clinical order or diagnosis.

Worked examples

- Pilot: test the same pattern set on one record from each of the four record types.
- Review: sample false positives and false negatives with a clinician before release.
- Release: deploy a versioned resource bundle and retain a rollback copy.

CONCLUSION

This assignment designed a morphology-aware normalisation layer for a hospital record search system. Regular expressions identify structured fields, finite automata validate fixed formats, morphological rules connect inflected forms, and a finite-state transducer relates surface words to lexical analyses. The Porter stemmer provides a useful baseline but can over-stem or under-stem clinical vocabulary. The generated synthetic corpus satisfies the minimum scale for records, record types and tokens, and the code execution produced measurable token, entity-hit and stemming counts.

The main design recommendation is a hybrid system. Exact patterns should protect identifiers and structured values. A maintained FST should handle known clinical morphology. A conservative fallback can support unknown English terms, while OCR correction and language identification should be added for noisy and code-mixed notes. The original text must always be preserved. Retrieval metrics requiring clinician relevance judgements remain unmeasured until a labelled query set is collected.